

Universidad Argentina de la Empresa

Facultad de Ingeniería y Ciencias Exactas (FAIN)

Programación III

Trabajo Práctico Obligatorio

Sistema de organización de pedidos
y planificación logística

Propuesta 2

Primera entrega

Informe parcial de diseño e implementación

Integrantes

Ferreira, Martin - Legajo 1187544
Rodriguez, Lautaro - Legajo 1134799

Índice

1. Selección de Propuesta....4
2. Resolución de Funcionalidades Obligatorias....4
2.1. Ejercicio 1....4
2.1.1. Descripción del Enunciado....4
2.1.2. Objetivo....4
2.1.3. Restricciones....4
2.1.4. Análisis del Modelo de Dominio (Los Datos)....5
2.1.5. Resolución del Ejercicio....5
2.2 Ejercicio 2 - Cálculo de la Ruta más Rápida....6
2.2.1. Descripción del Enunciado....6
2.2.2. Objetivo....6
2.2.3. Restricciones....6
2.2.4. Análisis del Modelo de Dominio (Los Datos)....6
2.2.5. Relaciones y Diseño....7
2.2.6. Resolución del Ejercicio....7
2.2.7. Test y Pruebas....8
2.2.8. Complejidad Temporal....9
2.3 Ejercicio 3....10
2.3.1. Descripción del Enunciado....10
2.3.2. Objetivo....10
2.3.3. Restricciones....10
2.3.4. Análisis del Modelo de Dominio (Los Datos)....10
2.3.5. Resolución del Ejercicio....11
3. Resolución de Funcionalidades Adicionales....12
3.1. Ejercicio A1 - Consultas de rutas entre todos los centros....12
3.1.1. Descripción del Enunciado....12
3.1.2. Objetivo....12
3.1.3. Restricciones....12
3.1.4. Análisis del Modelo de Dominio (Los Datos)....13
3.1.5. Relaciones y Diseño....13
3.1.6. Resolución del Ejercicio....13
3.1.7. Test y Pruebas....14
3.1.8. Complejidad Temporal....15
3.2. Ejercicio B1 - Recorrido con Múltiples Entregas....15
3.2.1. Descripción del Enunciado....15
3.2.2. Objetivo....15
3.2.3. Restricciones....15
3.2.4. Análisis del Modelo de Dominio (Los Datos)....16
3.2.5. Relaciones y Diseño....16
3.2.6. Resolución del Ejercicio....17
3.2.7. Test y Pruebas....18
3.2.8. Complejidad Temporal....20

3.3. Ejercicio B2 - Distribución de Cargas entre Vehículos.....20
3.3.1. Descripción del Enunciado.....20
3.3.2. Objetivo.....20
3.3.3. Restricciones.....21
3.3.4. Análisis del Modelo de Dominio (Los Datos).....21
3.3.5. Relaciones y Diseño.....21
3.3.6. Resolución del Ejercicio.....22
3.3.7. Test y Pruebas.....24
3.3.8. Complejidad Temporal.....25

1. Selección de Propuesta

La propuesta elegida fue la segunda. Es una organización que distribuye alimentos, medicamentos e insumos entre centros comunitarios, todos unidos por conexiones que tienen un sentido de circulación y un tiempo de viaje.

La problemática que tiene es la gestión y optimización. Se debe optimizar lo máximo posible tanto la organización de los pedidos como el cálculo de las rutas que se van a llevar a cabo para mandar la ayuda lo mejor y de más calidad posible.

2. Resolución de Funcionalidades Obligatorias

2.1. Ejercicio 1

2.1.1. Descripción del Enunciado

El ejercicio pide organizar y consultar los pedidos que recibe la organización. Cada pedido contiene un identificador, un centro de destino, una fecha límite, una prioridad y un peso total.

Se deben poder obtener los pedidos ordenados por fecha límite, comenzando por los más próximos, y por prioridad, comenzando por los más prioritarios. Cuando dos pedidos tienen la misma prioridad, se utiliza la fecha límite más cercana y, si también coinciden las fechas, el identificador menor. Además, se debe permitir buscar un pedido por su identificador.

2.1.2. Objetivo

El objetivo es facilitar la consulta de los pedidos y permitir encontrar uno en particular sin tener que recorrer siempre todos los elementos.

La entrada es un arreglo de pedidos y, para la búsqueda, un identificador. Las salidas son un nuevo arreglo ordenado según el criterio solicitado o el pedido correspondiente al identificador buscado. Si ese identificador no existe, la búsqueda devuelve null.

2.1.3. Restricciones

- Los identificadores de los pedidos deben ser únicos.
- El ordenamiento y la búsqueda deben implementarse sin utilizar los métodos de ordenamiento o búsqueda provistos por Java.
- El orden por fecha límite es ascendente.
- El orden por prioridad es descendente. Los empates se resuelven por fecha ascendente y luego por identificador ascendente.
- Deben contemplarse arreglos vacíos y búsquedas de identificadores inexistentes.

En la implementación también se valida que cada pedido tenga un centro de destino no vacío, una fecha límite y un peso positivo y finito.

2.1.4. Análisis del Modelo de Dominio (Los Datos)

El problema se modela mediante un arreglo de objetos Pedido. Cada objeto representa un pedido individual y reúne la información necesaria para ordenarlo, consultarlo y mostrarlo.

Las clases y sus responsabilidades son las siguientes:

- **Pedido:** representa los datos de un pedido: id, centroDestino, fechaLimite, prioridad y pesoTotal. La fecha se representa mediante LocalDate. Sus métodos permiten consultar los atributos y mostrar el pedido.
- **GestorPedidos:** conserva los pedidos ordenados por identificador. Permite consultar los pedidos por fecha o prioridad y buscar uno por su ID. También verifica que no existan identificadores repetidos.
- **OrdenamientoPedidos:** contiene la implementación de Merge-Sort y los criterios de comparación. Reúne las operaciones necesarias para ordenar los pedidos.
- **MainPrueba:** crea pedidos de ejemplo y muestra por consola los resultados de las consultas.
- **PruebasPedidos:** comprueba que el ordenamiento y la búsqueda produzcan los resultados esperados, incluyendo casos con arreglos vacíos y con un único pedido.

2.1.5. Resolución del Ejercicio

La estrategia seleccionada es divide y conquista, mediante Merge-Sort para ordenar y búsqueda binaria para consultar por identificador.

Merge Sort se encuentra en la clase OrdenamientoPedidos. El método ordenar copia el arreglo recibido e inicia el ordenamiento. El método mergeSort divide el segmento en dos mitades y ordena cada una recursivamente. Luego, el método merge combina ambas mitades en el orden establecido por el comparador.

La división termina cuando el segmento contiene un solo elemento o está vacío, porque en esos casos ya está ordenado. Durante la combinación, se comparan los elementos de ambas mitades y se incorpora primero el que corresponde según el criterio elegido.

Se seleccionó Merge-Sort porque mantiene un costo temporal de O(n log n) incluso en el peor caso. Quick-Sort también era una alternativa, pero puede alcanzar $O(n^2)$ cuando las particiones quedan muy desequilibradas.

Para buscar, el método buscarPorId de GestorPedidos utiliza una búsqueda binaria sobre pedidosPorId. Se compara el identificador buscado con el elemento central:

- Si coinciden, se devuelve ese pedido.
- Si el identificador buscado es menor, se continúa en la mitad izquierda.
- Si es mayor, se continúa en la mitad derecha.
- Si ya no quedan posiciones por revisar, se devuelve null.

En cada paso se descarta aproximadamente la mitad de los pedidos restantes. Por eso, la búsqueda tiene un costo temporal de O(log n) en el peor caso.

2.2 Ejercicio 2 - Cálculo de la Ruta más Rápida

2.2.1. Descripción del Enunciado

El siguiente ejercicio pide, dados una ubicación de origen y una de destino, obtener el recorrido de menor tiempo total entre ambas, mostrando las ubicaciones atravesadas y el tiempo que se tardó. Además, se deben poder deshabilitar las ubicaciones y volver a recalcular el tiempo y la ruta, informar cuando no exista una ruta disponible y poder visualizar todas las decisiones tomadas durante ese cálculo.

2.2.2. Objetivo

El objetivo, en síntesis, es minimizar la suma de los tiempos de las conexiones recorridas entre origen y destino. Recibe como entrada una red vial (grafo con ubicaciones y conexiones), un origen y un destino. La salida, si existe una ruta, es una lista ordenada de ubicaciones y tiempo total en minutos más otra lista de decisiones.

2.2.3. Restricciones

- Las conexiones son dirigidas.
- Los tiempos son enteros positivos.
- Solo se usan conexiones habilitadas.
- Puede no existir una ruta (un destino que está aislado, por ejemplo).

2.2.4. Análisis del Modelo de Dominio (Los Datos)

El problema se va a modelar como un grafo dirigido y ponderado, donde la red completa es el Grafo:

- Depósitos o Centros son los vértices.
- Las Conexiones viales con sentido son las aristas dirigidas.
- El tiempo de viaje es el peso de las aristas.

Las clases que componen a los datos y las responsabilidades son las siguientes:

- **TipoUbicación:** es un Enum que sirve únicamente para el tipo de ubicación que hay. En este sistema solo hay dos: Depósito o Centro Comunitario (DEPOSITO, CENTRO)
- **Ubicacion**: es una clase que da los datos de cada vértice: id, nombre y tipo, siendo el id el más importante de todos.
- **Conexion:** es la arista, tiene origen, destino, tiempo (peso) y si está o no habilitada.
- **RedVial:** guarda todas las ubicaciones en diccionarios y permite agregar, habilitar o deshabilitar una conexión. Es una clase importante ya que construye el Grafo que se le va a pasar a Dijkstra.
- **GrafoTDA:** es la interfaz con las operaciones de un grafo.
- **GrafoMatriz:** una clase que implementa al TDA para crear la matriz de adyacencia y un arreglo que relaciona cada posición de la matriz con el id del nodo. Cuando hay un 0 en la matriz significa que no hay una arista.
- **ResultadoRuta:** es el que orquesta la respuesta. Agrupa el origen, destino, si existe una ruta, el recorrido, el tiempo total y las decisiones de Dijkstra.

2.2.5. Relaciones y Diseño

La principal relación del proyecto a la de la clase RedVial la cual tiene Ubicacion y TipoUbicacion, tiene Conexion, para poder ver si está o no habilitada una conexión (arista), y por último está GrafoMatriz, la cual arma el grafo a partir de los datos que tiene RedVial para mandarselos a Dijkstra.

En lo que respecta a Decisiones la más importante es la Matriz de adyacencia, la cual permite consultar si hay o no una arista entre dos nodos y también poder saber su peso directamente. Esto se hace si son redes chicas, porque la complejidad es $O(n^2)$ y podría llegar a ser un problema si la red creciera mucho, sería una matriz enorme.

Por otro lado está la validación que tiene el modelo con los tiempos no positivos, los cuales son rechazados al momento de crear la conexión, entonces el algoritmo nunca podría recibir un grafo invalido. También, está el poder deshabilitar (y habilitar) una conexión en vez de borrarla. Cada vez que se calcula una ruta se reconstruye el grafo con las conexiones habilitadas, entonces si se deshabilita y se vuelve a recalcular es directo.

2.2.6. Resolución del Ejercicio

Sabiendo que el problema es de un camino mínimo desde origen en un grafo dirigido con pesos positivos, la mejor estrategia para esto es Greedy, más bien el algoritmo Dijkstra, ya que hay que encontrar, entre todos los caminos posibles del origen al destino, el que tiene la menor suma de pesos.

Dijkstra es un algoritmo Greedy: eso quiere decir que en cada paso elige, entre los vértices todavía no visitados o confirmados, el que tiene el menor tiempo conocido desde el origen y lo confirma como definitivo, sin volver a revisar la decisión.

La elección de Greedy se ve en la clase Dijkstra en el método seleccionarMinimo. Esto lo que hace es recorrer los nodos que no se visitaron, o sea vendrían a ser los pendientes, y devuelve el menor tiempo provisorio. Acá se puede ver el método:

```
private int seleccionarMinimo(GrafoTDA a, int v, Set<Integer> pendientes) {
  int minimo = -1;
  int pesoMinimo = Integer.MAX_VALUE;
  for (int n : pendientes) {
     if (a.existeArista(v, n) && a.pesoArista(v, n) < pesoMinimo) {
        pesoMinimo = a.pesoArista(v, n);
        minimo = n;
     }
  }
  return minimo;
}
```

La decisión que no tiene vuelta atrás es cuando el nodo que estaba en “pendientes” pasa a estar en “visitados”

Se elige Greddy porque al ser todos los tiempos positivos, si tenes un nodo con peso positivo, cualquiera hacia ese nodo tendría que pasar otro que pesa igual o más y sumas tramos positivos, así que nunca puede ser más corto. Esto lo garantiza Conexion en el modelo, ya que descarga tiempo menor o iguales a 0.

Otras alternativas que se sacaron fueron Floyd, ya que calculaba todos los pares, hacia $O(n^3)$ todo, y para un único origen era un exceso. También, se descartó usar Backtracking ya que ese sirve para listar todas las rutas, no para buscar la mínima solamente, era innecesario. Por esto mismo se termina eligiendo a Dijkstra.

2.2.7. Test y Pruebas

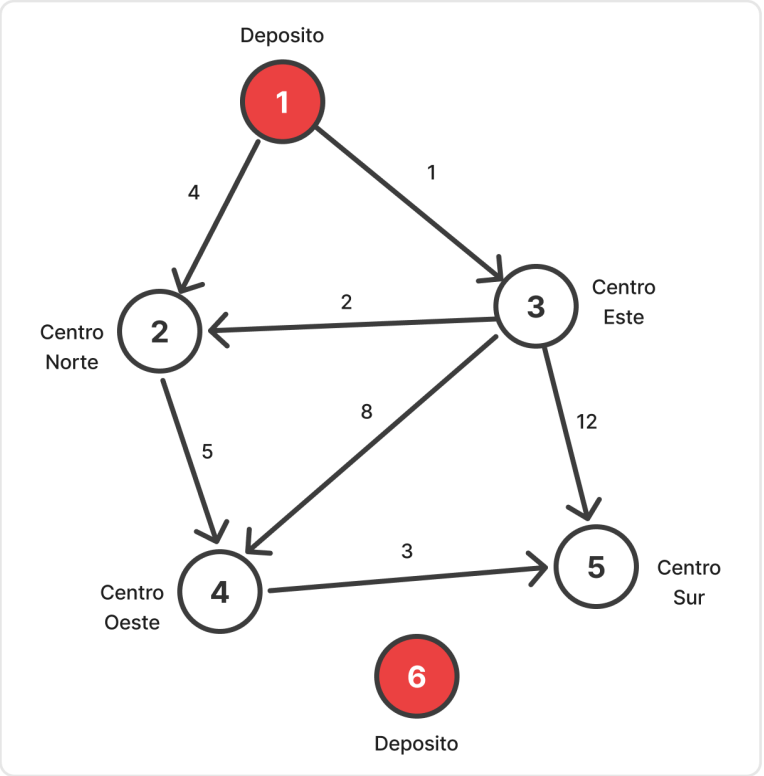
Todo comienza en la clase Test, la cual llama a Resolucion.calcularRutaMasRapida (red, origen, destino) e inicia el proceso.

Primero se valida que el origen y el destino existan, después si coinciden (origen == destino) para descartar ese caso particular de tiempo 0. Una vez que RedVial haya construido la red y GrafoMatriz hecho el grafo, se lo pasa a Dijkstra para resolver el problema del camino con el menor tiempo posible. Una vez terminado se reconstruye el recorrido que hizo y las decisiones para luego mostrarlas imprimiendolas.

Para hacer este Test, se creó una red de pruebas la cual demuestra visualmente un ejemplo de funcionamiento del algoritmo.

Paso	Confirma	2 Norte	3 Este	4 Oeste	5 Sur	6 Deposito
Inicio	1	4 (1)	1 (1)	∞	∞	∞
Vuelta 1	3	**3 (3)**	1 ✓	9 (3)	13 (3)	∞
Vuelta 2	2	3 ✓	1 ✓	**8 (2)**	13 (3)	∞
Vuelta 3	4	3 ✓	1 ✓	8 ✓	**11 (4)**	∞
Vuelta 4	5	3 ✓	1 ✓	8 ✓	11 ✓	∞
Vuelta 5	— (`-1` , corta)					∞

Red usada en la pruebas



2.2.8. Complejidad Temporal

Siendo n la cantidad de vértices y m la cantidad de aristas, la complejidad podría ser $O(n^2 + m \cdot n)$, pero dentro del método “ejecutar” en Dijkstra cada búsqueda es lineal, o sea (O(n), lo que termina dando un $O(n^3)$. El total entonces de una consulta es $O(n^3)$.

2.3 Ejercicio 3

2.3.1. Descripción del Enunciado

El ejercicio pide seleccionar los lotes que se cargarán en un vehículo con capacidad limitada. Cada lote disponible tiene un identificador, un peso entero positivo y un puntaje de prioridad entero no negativo.

La selección debe obtener la mayor prioridad total posible sin superar la capacidad del vehículo. Los lotes no pueden fraccionarse y cada uno puede cargarse una sola vez. Se deben mostrar los lotes seleccionados, el peso total, la prioridad obtenida y la capacidad restante.

2.3.2. Objetivo

El objetivo es maximizar la suma de las prioridades de los lotes seleccionados, respetando la capacidad máxima del vehículo.

La entrada es un arreglo de lotes disponibles y una capacidad máxima. La salida es una selección de lotes junto con su peso total, prioridad total y capacidad restante.

No es obligatorio llenar completamente el vehículo. Una selección puede dejar capacidad libre y aun así ser óptima si ninguna otra combinación válida obtiene una prioridad mayor.

2.3.3. Restricciones

- El peso de cada lote debe ser un número entero positivo.
- La prioridad de cada lote debe ser un número entero no negativo.
- Los lotes son indivisibles: se cargan completos o no se cargan.
- Cada lote puede seleccionarse como máximo una vez.
- El peso total seleccionado no puede superar la capacidad del vehículo.
- La capacidad se representa mediante un entero no negativo, se contempla el caso de capacidad cero.
- Cada posición del arreglo debe representar un lote distinto.
- La selección de cargas se resuelve por separado de la planificación del recorrido.

2.3.4. Análisis del Modelo de Dominio (Los Datos)

El problema se modela como una mochila. La capacidad del vehículo equivale a la capacidad de la mochila, mientras que cada lote equivale a un objeto que tiene un peso y un valor. En este caso, el valor es su prioridad.

La denominación 0/1 indica que cada lote tiene dos posibilidades: no seleccionarse o seleccionarse una vez.

Las clases y sus responsabilidades son las siguientes:

- **Lote:** representa un lote disponible mediante los atributos id, peso y prioridad. Valida que el peso sea positivo y que la prioridad no sea negativa.
- **SeleccionCargas:** contiene el algoritmo de programación dinámica. Su método seleccionar calcula la prioridad máxima y su método reconstruir identifica los lotes que permiten obtenerla.
- **ResultadoCarga:** agrupa los lotes seleccionados, el peso total, la prioridad total y la capacidad restante.
- **MainPrueba:** prepara un ejemplo y muestra la selección obtenida.
- **PruebasCargas:** verifica casos como una selección óptima conocida, capacidad cero, ausencia de lotes, lotes que no entran y empates de prioridad.

La capacidad del vehículo se recibe como un número entero. Para esta funcionalidad no se creó una clase Vehiculo.

2.3.5. Resolución del Ejercicio.

La estrategia seleccionada es programación dinámica, aplicada al problema de la mochila.

Esta estrategia permite resolver el problema a partir de soluciones más pequeñas. En lugar de calcular repetidamente todas las combinaciones, se guarda el mejor resultado para cada grupo de lotes y cada capacidad disponible.

El algoritmo recorre la matriz por filas. Para cada lote y capacidad se consideran los siguientes casos:

- **Primer lote:** si entra, se guarda su prioridad; si no entra, se guarda cero.
- **El lote actual no entra:** se conserva el resultado de la fila anterior para esa capacidad.
- **El lote actual entra:** se comparan dos alternativas: no cargarlo, conservando el resultado anterior, o cargarlo, sumando su prioridad al mejor resultado para la capacidad restante.

Cuando se evalúa cargar un lote, se consulta la fila anterior. Esto garantiza que ese mismo lote no pueda utilizarse nuevamente dentro de la combinación.

Al completar la tabla, la última fila en la columna de capacidad máxima contiene la mayor prioridad posible. Sin embargo, todavía es necesario identificar los lotes seleccionados.

Para hacerlo, el método “reconstruir” recorre la matriz desde la última fila hacia la primera. Si el valor actual es distinto del valor de la fila anterior, significa que incorporar ese lote mejoró la solución. Entonces se lo selecciona y se descuenta su peso de la capacidad pendiente.

Por ejemplo, con capacidad 8 y lotes de pesos 1, 4, 5 y 7, cuyas prioridades son 10, 3, 5 y 8, la solución selecciona los lotes de peso 1 y 7. El resultado tiene peso total 8, prioridad total 18 y capacidad restante 0.

Se eligió programación dinámica porque una decisión greedy no garantiza obtener la mejor combinación. Por ejemplo, con capacidad 10, seleccionar primero un lote de peso 6 y prioridad 9 impide cargar dos lotes de peso 5 y prioridad 8 cada uno,

que juntos alcanzarían una prioridad de 16.

3. Resolución de Funcionalidades Adicionales

Para esta etapa se van a realizar los tres ejercicios (uno del bloque A y dos del bloque B) adicionales, los cuales se eligen de manera opcional.

3.1. Ejercicio A1 - Consultas de rutas entre todos los centros

3.1.1. Descripción del Enunciado

El ejercicio pide preparar la red vial para responder numerosas consultas entre depósitos y centros comunitarios. Para cualquier par de ubicaciones, el sistema debe informar el tiempo mínimo de viaje y mostrar el recorrido que permite obtenerlo. Si no existe un camino en el sentido solicitado, debe indicarlo.

3.1.2. Objetivo

El objetivo es calcular de antemano los tiempos mínimos entre todos los pares de ubicaciones y guardar la información necesaria para reconstruir sus recorridos.

La entrada del cálculo inicial es la red vial. Luego, cada consulta recibe dos identificadores: origen y destino. La salida puede ser únicamente el tiempo mínimo o el resultado completo, con el origen, el destino, el tiempo total en minutos y la lista ordenada de ubicaciones atravesadas.

3.1.3. Restricciones

- Las conexiones son dirigidas. Que exista un camino de A hacia B no garantiza que exista uno de B hacia A.
- Los tiempos de viaje son enteros positivos y solo se consideran conexiones habilitadas.
- El origen y el destino deben pertenecer a la red utilizada.
- Si el origen y el destino coinciden, el tiempo es 0 y el recorrido contiene una sola ubicación.
- Si dos ubicaciones existen pero no hay camino entre ellas, el sistema devuelve un tiempo de -1 y un recorrido vacío.
- Si cambia la red, se debe llamar a recalcular para que las siguientes consultas reflejen ese cambio.

3.1.4. Análisis del Modelo de Dominio (Los Datos)

Se reutiliza la red vial del ejercicio 2. Cada Ubicación representa un vértice, cada Conexión habilitada representa una arista dirigida y su peso es el tiempo de viaje. La clase RedVial reúne esos datos y construye el grafo que recibe Floyd.
Para guardar los resultados se utilizan dos matrices:

- tiempos[i][j]: guarda el menor tiempo conocido para ir desde la ubicación de la posición i hasta la de la posición j.
- siguiente[i][j]: guarda la posición de la primera ubicación a la que hay que ir al salir de i para llegar a j por ese camino.

3.1.5. Relaciones y Diseño

La clase GestorRutas recibe una RedVial. Al crear el gestor, su método recalcular construye el grafo con las conexiones habilitadas y crea una instancia de Floyd. Así, las matrices quedan preparadas antes de la primera consulta.

El gestor ofrece dos operaciones. consultarTiempo lee el tiempo mínimo guardado. consultar también reconstruye el camino, convierte sus identificadores en objetos Ubicacion y devuelve un ResultadoConsultaRuta. Esta última clase agrupa los datos para que se puedan mostrar sin realizar cálculos adicionales.

La separación permite hacer varias consultas sobre el mismo gestor. Por ejemplo, después de consultar la ruta del depósito al Centro Sur, se puede consultar la del Centro Este al Centro Oeste utilizando las mismas matrices.

3.1.6. Resolución del Ejercicio

La estrategia elegida es programación dinámica mediante el algoritmo de Floyd. Se eligió porque el problema requiere tiempos mínimos entre todos los pares. También se podría ejecutar Dijkstra desde cada origen y guardar los resultados, pero Floyd resuelve directamente el problema de todos los pares.

Primero, el método inicializar prepara la matriz de tiempos con las rutas que se conocen sin pasar por ubicaciones intermedias. En la diagonal coloca 0, porque ir de una ubicación a sí misma no requiere viajar. Si existe una conexión directa, guarda su tiempo; en los demás casos coloca infinito. Para una conexión directa de i a j, el siguiente paso es el propio j.

Luego, el método ejecutar recorre las matrices mediante tres ciclos. El ciclo exterior elige una ubicación k que se permite usar como intermedia. Los dos ciclos interiores recorren cada origen i y cada destino j. Cuando existen los tramos de i a k y de k a j, se realiza la siguiente comparación:

```
long pasandoPorK = tiempos[i][k] + tiempos[k][j];

if (pasandoPorK < tiempos[i][j]) {
    tiempos[i][j] = pasandoPorK;
    siguiente[i][j] = siguiente[i][k];
}
```

La programación dinámica se reconoce en esa suma: los tiempos de ambos tramos ya se calcularon permitiendo las ubicaciones intermedias anteriores. Si pasar por k mejora el resultado, se actualiza el tiempo y se guarda el primer paso del camino de i a k. Si hay empate, se conserva el recorrido que ya se tenía.

Por ejemplo, en la red de prueba la conexión directa del Centro Este (3) al Centro Oeste (4) tarda 8 minutos. Al considerar al Centro Norte (2) como intermedio, Floyd encuentra que ir de 3 a 2 tarda 2 minutos y de 2 a 4 tarda 5. Como 2 + 5 = 7 mejora los 8 minutos anteriores, guarda el nuevo tiempo y registra que, para llegar de 3 a 4, el primer paso es ir a 2.

Finalmente, reconstruirRecorrido parte del origen y sigue los próximos pasos guardados hasta llegar al destino. En el ejemplo devuelve 3 → 2 → 4, sin volver a ejecutar Floyd.

3.1.7. Test y Pruebas

La clase PruebasRutas utiliza una red de cinco ubicaciones. Las primeras cuatro pruebas consultan el mismo gestor. En la última se modifica la red y se recalculan los resultados.

Caso	Consulta	Resultado obtenido
Ruta mínima	Del depósito central (1) al centro sur (5)	Recorrido 1 → 3 → 2 → 4 → 5. Tiempo: 11 minutos.
Otra consulta sin recalcular	Del Centro Este (3) al Centro Oeste (4).	Recorrido 3 → 2 → 4. Tanto la consulta completa como consultarTiempo devuelven 7 minutos.
Sin ruta disponible	Del Centro Sur (5) al Depósito Central (1).	No existe ruta en ese sentido. Se devuelve -1 y un recorrido vacío.
Origen igual a destino	Del Depósito Central (1) a sí mismo.	Recorrido [1]. Tiempo: 0 minutos.
Cambio en la red	Se deshabilita la conexión 2 → 4, se llama a recalcular y se vuelve a consultar de 1 a 5.	El nuevo recorrido es 1 → 3 → 4 → 5. El tiempo pasa de 11 a 12 minutos.

Las cinco pruebas pasaron. Se comprobó tanto el tiempo mínimo como el orden de las ubicaciones del recorrido.

3.1.8. Complejidad Temporal

Si N es la cantidad de ubicaciones, el cálculo de Floyd tiene un costo temporal de $O(N^3)$. Para cada una de las N ubicaciones intermedias se revisan N orígenes y N destinos. Cada comparación dentro de los tres ciclos cuesta O(1) porque accede directamente a posiciones de las matrices.

Una vez preparadas las tablas, consultar solo el tiempo tiene un costo esperado de O(1): se buscan las posiciones del origen y del destino en el mapa y se lee una celda. Reconstruir el recorrido cuesta O(L), donde L es la cantidad de ubicaciones que contiene el camino, ya que se sigue un próximo paso por vez.

3.2. Ejercicio B1 - Recorrido con Múltiples Entregas

3.2.1. Descripción del Enunciado

El ejercicio pide planificar el recorrido de un vehículo que sale de un depósito, debe pasar por una serie de centros seleccionados, como máximo 8, y al terminar tiene que regresar al mismo depósito. Cada centro aparece una sola vez como parada.

Se conocen los tiempos mínimos entre las ubicaciones y se debe encontrar el orden de visita que tenga el menor tiempo total. Entre dos paradas el vehículo puede atravesar otras ubicaciones, por lo que no alcanza con mirar las conexiones directas. Además, se debe informar cuando no exista un recorrido posible.

3.2.2. Objetivo

El objetivo es minimizar el tiempo total del recorrido, que es la suma de los tiempos mínimos de cada tramo entre paradas consecutivas, incluyendo el regreso al depósito. En síntesis, es el problema del viajante aplicado a las paradas elegidas.

La entrada es una red vial, el identificador del depósito y una lista con los identificadores de los centros a visitar. La salida, si existe un recorrido, es el orden de visita, las ubicaciones que atraviesa cada tramo, el tiempo de cada uno y el tiempo total. También se muestran los nodos explorados y las ramas podadas. Si no existe un recorrido, se informa.

3.2.3. Restricciones

- El recorrido empieza y termina en el mismo depósito.
- Se pueden seleccionar como máximo 8 centros.
- Un centro no puede estar repetido en la lista de paradas.
- La ubicación de partida debe ser de tipo depósito y las paradas deben ser de

tipo centro.
- Un tramo puede atravesar ubicaciones intermedias.
- Las conexiones son dirigidas, por lo que el tiempo de ida y el de vuelta pueden ser distintos.
- Los tiempos son enteros positivos.
- Solo se usan conexiones habilitadas.
- Puede no existir un recorrido, por ejemplo si no se puede llegar a un centro o si desde él no se puede regresar al depósito.

3.2.4. Análisis del Modelo de Dominio (Los Datos)

El problema se sigue modelando como un grafo dirigido y ponderado, pero el algoritmo no trabaja directamente sobre ese grafo. A partir de la red se arma una matriz de tiempos mínimos que contiene solamente las paradas del recorrido. La posición 0 es el depósito y las posiciones siguientes son los centros.

Las clases y sus responsabilidades son las siguientes:

- **Ubicacion y TipoUbicacion:** representan el depósito y los centros. Se reutilizan de los ejercicios anteriores. No fue necesario crear una clase para cada tipo de ubicación.
- **RedVial:** contiene las ubicaciones y las conexiones, y construye el grafo con las conexiones habilitadas.
- **Floyd:** se reutiliza del ejercicio A1. Calcula los tiempos mínimos entre todos los pares de ubicaciones y permite reconstruir por dónde pasa cada tramo.
- **BacktrackingRecorrido:** contiene el algoritmo. Recibe la matriz de tiempos y busca el recorrido de menor tiempo total. También cuenta los nodos explorados y las ramas podadas.
- **Resolucion:** valida los datos, arma la matriz de tiempos, ejecuta el algoritmo y muestra el resultado.
- **Test:** crea las redes de prueba y ejecuta los casos.

Para esta funcionalidad no se utiliza la clase Vehiculo, ya que hay un único vehículo y sus datos no influyen en el recorrido.

3.2.5. Relaciones y Diseño

La relación principal parte de Resolucion, que le pide a RedVial el grafo y se lo pasa a Floyd. Con los tiempos calculados arma la matriz y se la entrega a BacktrackingRecorrido. Cuando el algoritmo termina, Resolucion vuelve a usar Floyd para mostrar las ubicaciones que atraviesa cada tramo.

La decisión más importante es que BacktrackingRecorrido no conozca las ubicaciones ni sus identificadores: solo recibe una matriz donde tiempos[i][j] es el tiempo mínimo de la parada i a la parada j. Resolucion es quien traduce esas posiciones a ubicaciones cuando muestra el resultado. Así, el algoritmo queda separado del modelo.

Se eligió Floyd porque hacen falta los tiempos entre todos los pares de paradas, a diferencia del ejercicio 2, donde había un único origen. Se calcula una sola vez y después cada consulta a la tabla es directa. Cuando no hay ruta, Floyd devuelve -1, que es el mismo valor que utiliza el algoritmo como SIN_CAMINO.

Por último, la validación se hace antes de ejecutar el algoritmo. Si hay más de 8 centros, si uno está repetido, si una ubicación no existe o si no es del tipo correcto, los datos se rechazan con una excepción.

3.2.6. Resolución del Ejercicio

La estrategia seleccionada es Backtracking con poda, ya que hay que elegir un orden entre todos los posibles y asegurar que sea el mejor. Con n centros existen n! órdenes de visita, que para 8 centros son 40.320.

El recorrido se arma etapa por etapa: la etapa 0 siempre es el depósito, la etapa 1 es el primer centro, la etapa 2 es el segundo, y así sucesivamente. En cada etapa se prueba cada centro posible y después se vuelve atrás para probar el siguiente.

A diferencia de un Backtracking que solo muestra cada solución, acá se guarda la mejor encontrada hasta el momento. Cada solución completa que mejora el tiempo reemplaza a la anterior, y por eso el recorrido se copia: el arreglo recorridoActual se sigue modificando mientras se exploran otras ramas.

La estrategia se ve en la clase BacktrackingRecorrido, en el método backtracking:

```
private void backtracking(int etapa, int tiempoActual) {
    int paradaAnterior = recorridoActual[etapa - 1];
    for (int centro = 1; centro < tiempos.length; centro++) {
        if (!visitados[centro] && tiempos[paradaAnterior][centro] != SIN_CAMINO &&
tiempos[centro][0] != SIN_CAMINO) {
            recorridoActual[etapa] = centro;
            visitados[centro] = true;
            nodosExplorados++;
            int tiempoNuevo = tiempoActual + tiempos[paradaAnterior][centro];

            if (tiempoNuevo + tiempos[centro][0] < mejorTiempo) {
                if (etapa == tiempos.length - 1) {
                    evaluarSolucion(tiempoNuevo);
                } else {
                    backtracking(etapa + 1, tiempoNuevo);
                }
            } else {
                ramasPodadas++;
            }
```

```
            visitados[centro] = false;
        }
    }
}
```

Las partes del esquema general se reconocen de la siguiente manera:

- **Valores posibles:** el for recorre los centros y solo se prueban los que no fueron visitados, se pueden alcanzar desde la parada anterior y tienen regreso al depósito.
- **Agregar:** se coloca el centro en recorridoActual, se marca como visitado y se suma el tiempo del tramo.
- **Factible:** se verifica que la rama todavía pueda mejorar a la mejor solución.
- **Es solución:** cuando la etapa es la última, ya se ubicaron todos los centros y se llama a evaluarSolucion, que guarda el recorrido.
- **Eliminar:** el centro se desmarca siempre, sin importar si la rama se podó o no.

La poda consiste en sumar al tiempo parcial el regreso directo desde el centro actual hasta el depósito. Si ese valor iguala o supera al mejor tiempo encontrado, la rama no puede mejorar y se abandona. Es correcta porque la matriz tiene tiempos mínimos: volver al depósito pasando por los centros que faltan nunca puede costar menos que volver directamente.

Se descartó Greedy porque elegir siempre el centro más cercano no garantiza el mejor recorrido. Por ejemplo, con un depósito D y dos centros A y B, sean los tiempos D→A = 1, D→B = 2, A→B = 4, B→A = 1, A→D = 2 y B→D = 3. Si se va primero al más cercano, el recorrido D, A, B, D tarda 8 minutos, mientras que D, B, A, D tarda 5.

También se descartó una fuerza bruta sin poda, porque recorrería siempre el árbol completo. Por último, Programación Dinámica podría resolver el problema guardando resultados por subconjunto de centros, pero con un máximo de 8 centros no hace falta esa tabla adicional y Backtracking resulta viable.

3.2.7. Test y Pruebas

Todo comienza en la clase Test, la cual llama a Resolucion.resolver(red, idDeposito, idsCentros). Primero se validan los datos y se arma la lista de paradas, con el depósito en la posición 0. Después se construye la matriz de tiempos mínimos con Floyd y se ejecuta el Backtracking. Por último se muestran el orden de visita, los tramos, el tiempo total y los contadores.

Se utilizaron los siguientes casos de prueba:

Caso	Qué se prueba	Resultado

1	Cuatro centros cargados desordenados	Norte, Este, Sur, Oeste: 16 minutos, 43 nodos y 17 ramas podadas
2	Tres centros	Norte, Este, Sur: 16 minutos, 12 nodos y 5 ramas podadas
3	Un centro, con tramos que pasan por otras ubicaciones	7 minutos
4	Sin centros	0 minutos
5	Un centro inalcanzable	No existe recorrido
6	Ocho centros, el máximo permitido	Del Centro 2 al Centro 9 en orden: 9 minutos, 680 nodos y 482 ramas podadas
7	Más de ocho centros	Datos rechazados
8	Un centro repetido	Datos rechazados
9	Partida desde una ubicación que no es un depósito	Datos rechazados
10	Un centro sin regreso al depósito	No existe recorrido

Para mostrar el funcionamiento se detalla el Caso 2, con las paradas 0 = Deposito Central, 1 = Centro Norte, 2 = Centro Este y 3 = Centro Sur. La matriz de tiempos mínimos es la siguiente, donde la fila es el origen y la columna es el destino:

	0	**1**	**2**	**3**
0	0	2	3	7
1	5	0	2	6
2	3	5	0	4
3	8	7	6	0

Por ejemplo, 0→1 vale 2 porque pasa por Centro Plaza y 3→0 vale 8 porque pasa por Centro Oeste.

3.2.8. Complejidad Temporal

Siendo n la cantidad de centros y V la cantidad de ubicaciones de la red, en la primera etapa hay n opciones, en la segunda n-1, y así sucesivamente. Cada llamada recorre los centros con un for, por lo que la recurrencia es $T(n) = n \cdot T(n-1) + O(n)$, y el Backtracking resulta $O(n \cdot n!)$. Tomando a = n y b = 1 como en las 8 reinas, se obtiene la cota $O(n^n)$, que es menos ajustada pero también válida.

La poda no cambia el peor caso, pero en la práctica descarta muchas ramas. En el Caso 6 se exploraron 680 nodos de los 109.600 que tiene el árbol completo.

Armar la matriz tiene un costo de $O(V^3)$ por Floyd. Una vez calculada, cada consulta a la matriz es $O(1)$ y, además, copiar una solución que mejora cuesta $O(n)$. El total de una consulta es entonces $O(V^3 + n \cdot n!)$.

3.3. Ejercicio B2 - Distribución de Cargas entre Vehículos

3.3.1. Descripción del Enunciado

El ejercicio pide repartir un conjunto de lotes entre varios vehículos. Cada lote tiene un peso y una lista de vehículos habilitados para llevarlo, mientras que cada vehículo tiene una capacidad máxima.

Se deben distribuir todos los lotes, sin partirlos ni repetirlos, sin superar la capacidad de ningún vehículo y usando únicamente vehículos habilitados. Se debe mostrar una distribución válida o informar que no existe.

3.3.2. Objetivo

El objetivo es encontrar una distribución válida de todos los lotes. A diferencia de los ejercicios anteriores, no se busca optimizar un valor: alcanza con encontrar una sola solución.

La entrada es un arreglo de lotes, un arreglo de vehículos y, para cada lote, la lista de identificadores de los vehículos habilitados. La salida, si existe una distribución, son los lotes que lleva cada vehículo junto con su peso cargado y su capacidad restante. También se muestran las asignaciones probadas, las ramas podadas y las asignaciones deshechas. Si no existe una distribución, se informa.

3.3.3. Restricciones

- Todos los lotes deben quedar asignados.
- Cada lote va completo en un único vehículo.
- Un lote solo puede ir en un vehículo habilitado para llevarlo.
- La suma de los pesos de un vehículo no puede superar su capacidad.
- Los pesos de los lotes y las capacidades de los vehículos son enteros positivos.
- Debe haber una lista de vehículos habilitados por cada lote.
- Los vehículos indicados en esas listas deben existir.
- Puede no existir una distribución válida.

3.3.4. Análisis del Modelo de Dominio (Los Datos)

El problema se modela mediante arreglos. Cada lote y cada vehículo se identifican por su posición, y la solución se guarda en un arreglo asignacion, donde asignacion[lote] es la posición del vehículo que lleva ese lote.

Las compatibilidades no se guardan dentro de Lote. Se reciben aparte en un int[][], donde la fila i tiene los identificadores de los vehículos habilitados para el lote i. Por ejemplo, {{1, 2}, {1, 3}} significa que el primer lote puede ir en los vehículos 1 o 2, y el segundo en los vehículos 1 o 3. Después se convierten en una matriz boolean[][] habilitado, que indica directamente si un vehículo puede llevar un lote.

Las clases y sus responsabilidades son las siguientes:

- **Lote:** se reutiliza del ejercicio 3. Tiene id, peso y prioridad, y valida que el peso sea positivo. Se le agregó un constructor que recibe solo id y peso, ya que en este ejercicio la prioridad no se utiliza.
- **Vehiculo:** es una clase nueva que representa un vehículo mediante id y capacidad. Valida que la capacidad sea positiva.
- **BacktrackingDistribucion:** contiene el algoritmo. Recibe los pesos, las capacidades y la matriz de vehículos habilitados, y busca una asignación válida. También cuenta las asignaciones probadas, las ramas podadas y las asignaciones deshechas.
- **Resolucion:** valida los datos, arma los arreglos, hace un chequeo previo, ejecuta el algoritmo y muestra el resultado.
- **Test:** ejecuta los casos de prueba.

3.3.5. Relaciones y Diseño

La relación principal parte de Resolucion, que recibe los lotes y los vehículos, y arma con ellos los arreglos que necesita BacktrackingDistribucion. El algoritmo no conoce las clases Lote ni Vehiculo, solo trabaja con posiciones. Cuando termina, Resolucion traduce esas posiciones a lotes y vehículos para mostrar el resultado.

La clase Vehiculo se ubica en el paquete modelo porque representa un dato general del sistema y no algo exclusivo de este ejercicio.

Otra decisión importante es el chequeo previo que se hace en Resolucion antes de ejecutar el Backtracking. Con este chequeo se descartan los casos que seguro no tienen solución:

- Un lote no tiene ningún vehículo habilitado.
- Un lote pesa más que todos sus vehículos habilitados.
- El peso total de los lotes supera la capacidad total de los vehículos.

Si se cumple alguno, se informa el motivo y no se ejecuta el algoritmo. Pasar el chequeo no garantiza que exista una solución, como se ve en el Caso 7 de las pruebas.

3.3.6. Resolución del Ejercicio

La estrategia seleccionada es Backtracking con poda. El problema se resuelve lote por lote: cada etapa corresponde a un lote y los valores posibles son

los vehículos habilitados para llevarlo. Si una decisión lleva a un callejón sin salida, se deshace y se prueba con el siguiente vehículo.

Como alcanza con una sola solución, el método devuelve un boolean y termina apenas la encuentra, igual que en las 8 reinas. La estrategia se ve en la clase BacktrackingDistribucion, en el método backtracking:

```
private boolean backtracking(int lote) {
   boolean solucion = false;
   int vehiculo = 0;
   while (vehiculo < capacidadRestante.length && !solucion) {
      if (habilitado[lote][vehiculo]) {
         asignacionesProbadas++;
         asignacion[lote] = vehiculo;
         capacidadRestante[vehiculo] = capacidadRestante[vehiculo] - pesos[lote];

         if (capacidadRestante[vehiculo] >= 0) {
            if (lote == pesos.length - 1) {
               solucion = true;
            } else if (hayLotePendienteSinLugar(lote + 1)) {
               ramasPodadas++;
            } else {
               solucion = backtracking(lote + 1);
            }
         }

         if (!solucion) {
            capacidadRestante[vehiculo] = capacidadRestante[vehiculo] + pesos[lote];
            asignacion[lote] = SIN_ASIGNAR;
            asignacionesDeshechas++;
         }
      }
      vehiculo++;
   }
   return solucion;
}
```

Las partes del esquema general se reconocen de la siguiente manera:

- **Valores posibles:** el while recorre los vehículos y solo se prueban los habilitados para el lote, mientras no se haya encontrado una solución.
- **Agregar:** se asigna el lote al vehículo y se resta su peso de la capacidad restante.
- **Factible:** la capacidad restante debe quedar mayor o igual a 0. Como primero se agrega y después se verifica, el chequeo se hace sobre la capacidad ya descontada.
- **Es solución:** si el lote asignado es el último, todos los lotes tienen vehículo.
- **Eliminar:** si no se encontró una solución, se devuelve el peso al vehículo y el

lote vuelve a quedar sin asignar. Cuando sí se encontró, no se elimina para no perder la asignación.

La poda se hace en el método hayLotePendienteSinLugar, que revisa los lotes que todavía no se asignaron. Si alguno no entra en ninguno de sus vehículos habilitados, la rama se abandona. Es correcta porque a medida que se baja en el árbol solo se agregan lotes, por lo que la capacidad restante nunca aumenta: si un lote pendiente no entra ahora, tampoco va a entrar más adelante. Por lo tanto, cortar esa rama no pierde soluciones.

No se usó como poda la comparación entre el peso pendiente y la capacidad restante total. Al asignar un lote ambos valores bajan lo mismo, por lo que la diferencia entre ellos no cambia durante el Backtracking. Por eso esa comparación solo sirve en el chequeo previo.

Se descartó Greedy porque, al no poder volver atrás, puede fallar aunque exista una solución. Por ejemplo, con vehículos de capacidad 10 y 6, y lotes de peso 6, 5 y 5, poner cada lote en el primer vehículo donde entra deja el lote 1 en el vehículo de 10 y el lote 2 en el de 6. Así, el lote 3 ya no entra en ninguno. Sin embargo, sí existe una distribución: el lote 1 en el vehículo de 6 y los otros dos en el de 10.

También se descartó Programación Dinámica. En la mochila del ejercicio 3 hay una sola capacidad, pero acá el estado tendría que guardar la capacidad restante de cada vehículo. La tabla tendría n × C1 × C2 × ... × Cm celdas y crecería exponencialmente con la cantidad de vehículos. Además, no hay un valor que optimizar.

3.3.7. Test y Pruebas

Todo comienza en la clase Test, la cual llama a Resolucion.resolver(lotes, vehiculos, vehiculosHabilitados). Primero se validan los datos y se arman los arreglos de pesos y capacidades junto con la matriz de vehículos habilitados. Después se hace el chequeo previo y, si se supera, se ejecuta el Backtracking. Por último se muestra la distribución y los contadores.

Los contadores miden lo siguiente:

- **Asignaciones probadas:** cada vez que se coloca un lote en un vehículo habilitado, entre o no entre.
- **Ramas podadas:** cada vez que el lote entra en el vehículo, pero algún lote pendiente queda sin lugar.
- **Asignaciones deshechas:** cada vez que se quita un lote de un vehículo.

Se utilizaron los siguientes casos de prueba, donde V es vehículo y L es lote:

Caso	Qué se prueba	Resultado
1	Solución directa	L1 y L2 en V1, L3 en V2: 3 probadas, 0 podadas y 0 deshechas
2	Caso donde Greedy falla (V de 10 y 6, L de 6, 5 y 5)	L2 y L3 en V1, L1 en V2: 6 probadas, 1 podada y 3 deshechas
3	Lotes con distintas compatibilidades	L2 y L4 en V1, L1 en V2, L3 en V3: 10 probadas, 1 podada y 6 deshechas
4	Un lote sin vehículos habilitados	No existe, se descarta en el chequeo previo
5	Un lote más pesado que todos sus vehículos habilitados	No existe, se descarta en el chequeo previo
6	Peso total mayor a la capacidad total	No existe, se descarta en el chequeo previo
7	Capacidad total suficiente, pero sin solución (V de 5 y 5, L de 4, 3 y 3)	No existe, se determina con el Backtracking: 6 probadas, 2 podadas y 6 deshechas
8	Lote con peso 0	Datos rechazados
9	Vehículo con capacidad 0	Datos rechazados
10	Compatibilidad con un vehículo inexistente	Datos rechazados

Para mostrar el funcionamiento se detalla el Caso 3. Los vehículos son V1 = 8, V2 = 6 y V3 = 4. Los lotes son L1 = 5 (V1 o V2), L2 = 4 (V1 o V3), L3 = 4 (V2 o V3) y L4 = 4 (V1 o V2). Las capacidades restantes se muestran como [V1, V2, V3] y arrancan en [8, 6, 4].

Paso	Asignación	Capacidades	Decisión
1	L1 → V1	[3, 6, 4]	Entra y los pendientes tienen lugar, sigue
2	L2 → V1	[-1, 6, 4]	No entra, se deshace
3	L2 → V3	[3, 6, 0]	Entra y los pendientes tienen lugar, sigue
4	L3 → V2	[3, 2, 0]	Entra, pero L4 queda sin lugar: poda, se deshace
5	L3 → V3	[3, 6, -4]	No entra, se deshace. Se vuelve atrás y se deshacen L2 → V3 y L1 → V1
6	L1 → V2	[8, 1, 4]	Entra y los pendientes tienen lugar, sigue
7	L2 → V1	[4, 1, 4]	Entra y los pendientes tienen lugar, sigue
8	L3 → V2	[4, -3, 4]	No entra, se deshace
9	L3 → V3	[4, 1, 0]	Entra y el pendiente tiene lugar, sigue
10	L4 → V1	[0, 1, 0]	Entra y es el último lote: solución

El resultado es L1 en V2, L2 en V1, L3 en V3 y L4 en V1, con 10 asignaciones probadas, 1 rama podada y 6 asignaciones deshechas. Estos valores coinciden con los que muestra el programa.

3.3.8. Complejidad Temporal

Siendo n la cantidad de lotes y m la cantidad de vehículos, cada llamada prueba hasta m vehículos y por cada uno puede hacer una llamada con un lote menos. Además, la poda recorre los lotes pendientes y sus vehículos, lo que cuesta $O(n \cdot m)$. La recurrencia es entonces $T(n) = m \cdot T(n-1) + O(n \cdot m)$.

Con la fórmula para la reducción por resta, tomando a = m, b = 1 y k = 1, y siendo m > 1, se obtiene $T(n) \in \Theta(a^{(n \text{ div } b)})$, que da $O(m^n)$.

La poda y el corte al encontrar la primera solución no cambian el peor caso, pero en la práctica reducen los nodos visitados. El chequeo previo y el armado de los arreglos cuestan $O(n \cdot m)$, que es despreciable frente a m^n. El total es entonces $O(m^n)$.